Carleton University
Department of Systems and Computer Engineering

SYSC 3303A

Real-Time Concurrent Systems — Winter 2026

Firefighting Drone Swarm

Final Project Report

Group 5

Name	Student ID
Faraaz	101298165
Sabateesh	101259947
Aymen	101326354
Ibrahim	101235067

April 5, 2026

Contents

1 Introduction **3**

2 Breakdown of Responsibilities **3**
2.1 Iteration 1 — Basic Communication . . . 3
2.2 Iteration 2 — Scheduling & Drone State Machines . . . 3
2.3 Iteration 3 — Multiple Drones & Distributed Execution . . . 4
2.4 Iteration 4 — Fault Handling . . . 4
2.5 Iteration 5 — Capacity Limits, GUI & Metrics . . . 4

3 Diagrams **6**
3.1 UML Class Diagram . . . 6
3.2 State Machine Diagram — Scheduler . . . 7
3.3 State Machine Diagram — Drone . . . 7
3.4 State Machine Diagram — Drone (Updated with Faults) . . . 8
3.5 Sequence Diagram — Deploy Drone . . . 10
3.6 Sequence Diagram — Initialization . . . 11
3.7 Sequence Diagram — Refill . . . 12
3.8 Sequence Diagrams — Error Scenarios . . . 12
3.9 Timing Diagrams for the Scheduler . . . 13

4 Fault Handling **14**
4.1 Fault Types . . . 14
4.2 Fault Injection Methods . . . 14
4.2.1 CSV Input File . . . 14
4.2.2 GUI Injection . . . 14
4.3 Fault Detection Mechanisms . . . 14
4.4 Scheduler Fault Response . . . 15
4.5 Drone Fault Behavior . . . 15
4.5.1 Transient Faults (Drone Stuck, Arrival Sensor) . . . 15
4.5.2 Hard Faults (Nozzle Jammed) . . . 16

5 Detailed Setup and Test Instructions **16**
5.1 Prerequisites . . . 16
5.2 Running the Simulation . . . 16
5.3 Testing Fault Scenarios . . . 16
5.3.1 CSV-Based Fault Injection . . . 16
5.3.2 GUI-Based Fault Injection . . . 17
5.4 Unit Tests . . . 17
5.5 Test Input Files . . . 17

6 Results from Measurements **17**

7 Design Reflection **18**
7.1 What We Like 18
7.2 What Should Be Redone 19

1 Introduction

This report documents the design, implementation, and testing of a firefighting drone swarm control system developed over five iterations for SYSC 3303A. The system consists of three distributed subsystems communicating over UDP:

1. **Scheduler** — Central coordinator that receives fire incident requests, dispatches drones, monitors status, detects faults, and manages mission queues.
2. **Drone Subsystem** — Manages multiple drone threads, each implementing a state machine for flight, agent delivery, refilling, and fault handling.
3. **Fire Incident Subsystem** — Reads fire events from a CSV input file and sends them to the Scheduler with configurable timing and fault injection.

Each subsystem runs as a separate Java process, communicating exclusively via `DatagramSocket` (UDP). The system operates in simulated real-time with configurable parameters including the number of drones, number of zones, travel speed, agent capacity, and simulation speed. The final iteration supports 20 drones across 5+ fire zones with full fault handling, a real-time GUI, and automatic performance metrics collection.

2 Breakdown of Responsibilities

2.1 Iteration 1 — Basic Communication

Member	Responsibilities
Faraaz (101298165)	Documentation, diagrams, fixing bugs and code restructuring
Sabateesh (101259947)	FireIncidentSubsystem, FireIncidentSubsystem tests
Ibrahim (101235067)	Scheduler, SchedulerServer
Aymen (101326354)	DroneSubsystem, Drone

2.2 Iteration 2 — Scheduling & Drone State Machines

Member	Responsibilities
Faraaz (101298165)	Scheduling logic, work on GUI
Sabateesh (101259947)	Main GUI development, tests for DroneStateMachine
Ibrahim (101235067)	Making diagrams, state machine planning
Aymen (101326354)	Drone state transitions, GUI interaction setup

2.3 Iteration 3 — Multiple Drones & Distributed Execution

Member	Responsibilities
Faraaz (101298165)	UDP communication between Scheduler and subsystems, UDP message test class, updates to GUI based on Scheduler datagrams
Sabateesh (101259947)	UML sequence and class diagrams, GUI updates to information based on fire info, display drone path
Aymen (101326354)	Update Drone Subsystem UDP communications, packet handling and registration with Scheduler
Ibrahim (101235067)	GUI visual changes and reorganization, updated GUI to show drone position, animations for visual feedback

2.4 Iteration 4 — Fault Handling

Member	Responsibilities
Faraaz (101298165)	Implemented drone faults and drone fault detection, CRC checksum redundancy in UDP messages, marking drones as faulty when detected
Sabateesh (101259947)	GUI changes, updated GUI to show drone fault handling
Aymen (101326354)	Diagrams, updated drone state machine, update Drone Subsystem UDP communications, packet handling and registration with Scheduler
Ibrahim (101235067)	Drone fault handling, returning faulty drones to base station, redistributing tasks when drone failure occurs

2.5 Iteration 5 — Capacity Limits, GUI & Metrics

Member	Responsibilities
Faraaz (101298165)	Added simulation metrics recording, fixed fault detection for nozzle jam and stuck mid-flight, fault info displayed on GUI, drone real-time location
Sabateesh (101259947)	Fault fixes, final report
Aymen (101326354)	Timing diagrams
Ibrahim (101235067)	Timing diagrams

3 Diagrams

3.1 UML Class Diagram

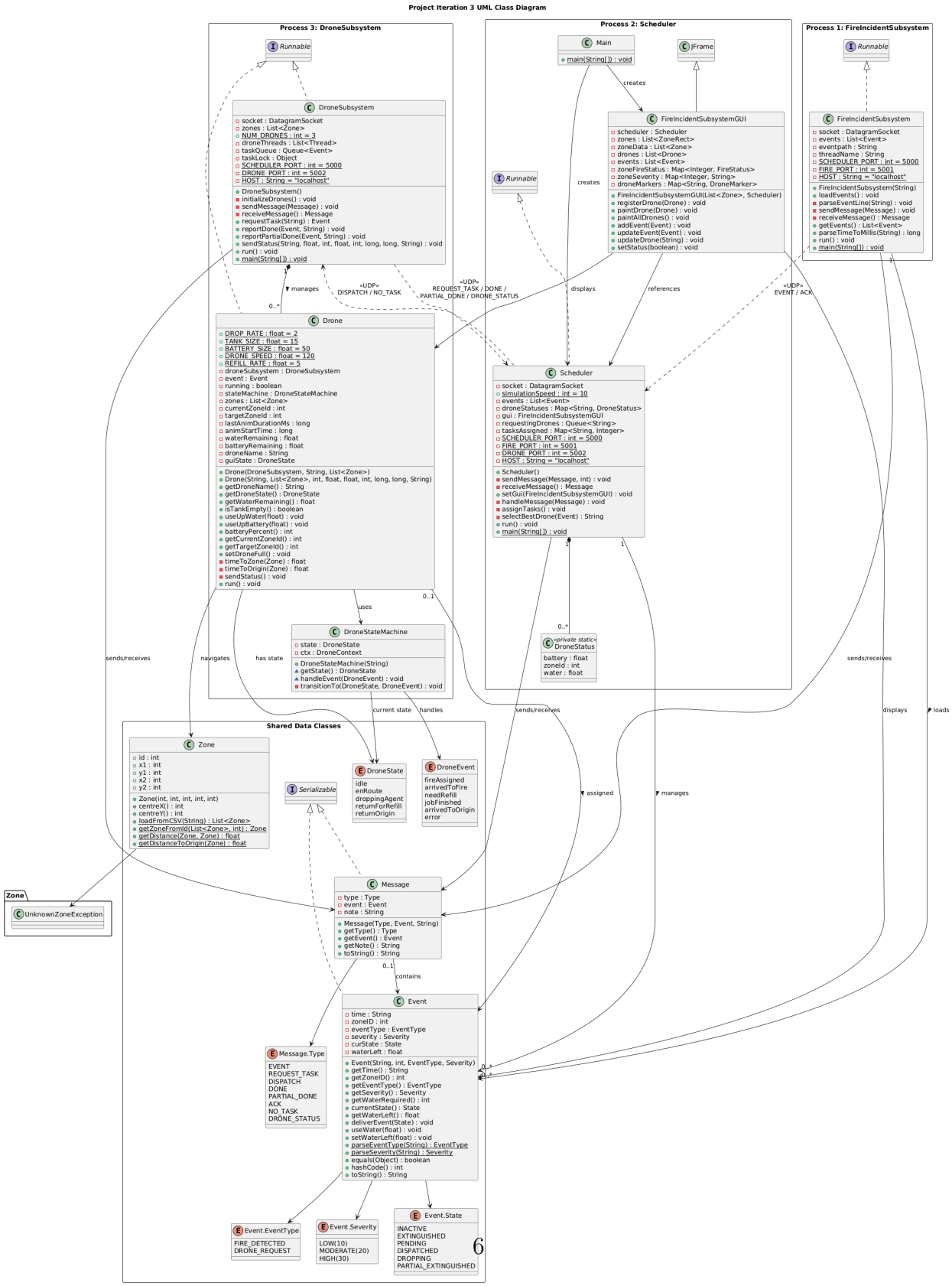


Figure 1: UML Class Diagram for the Scheduler, Drone Subsystem, and Fire Incident Sub-

3.2 State Machine Diagram — Scheduler

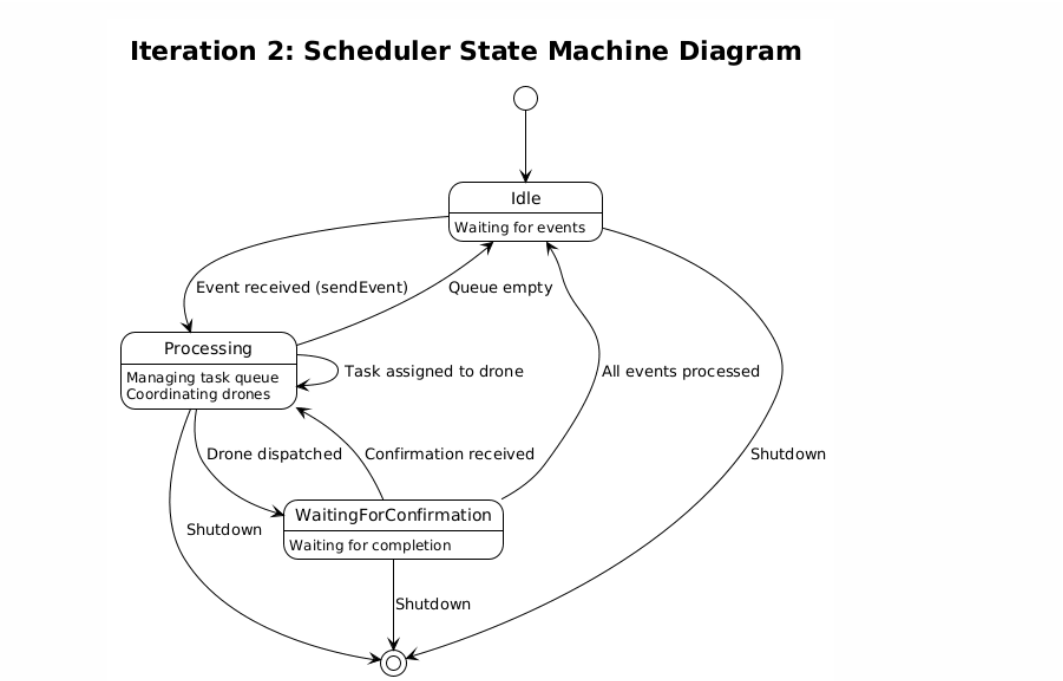


Figure 2: Scheduler State Machine Diagram.

3.3 State Machine Diagram — Drone

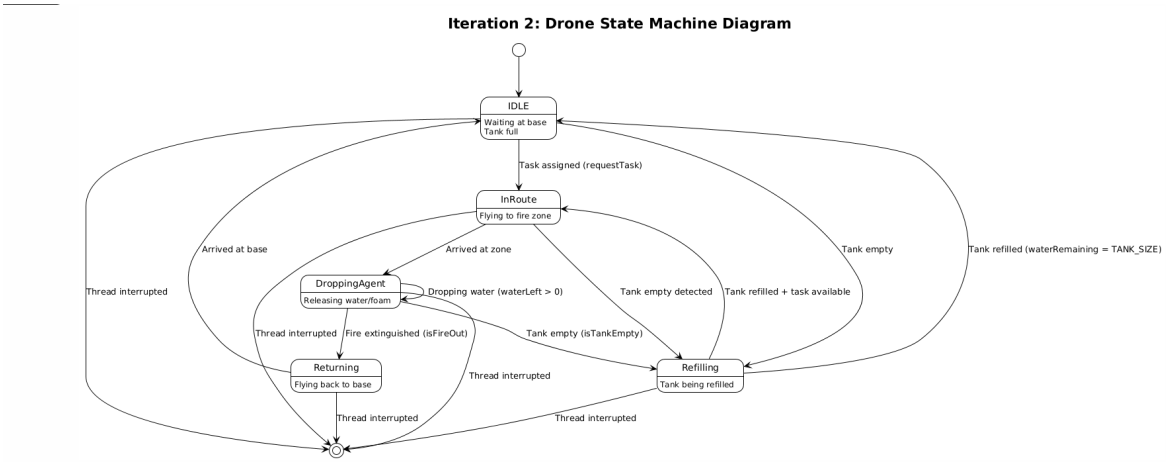


Figure 3: Drone State Machine Diagram (Iteration 2).

3.4 State Machine Diagram — Drone (Updated with Faults)

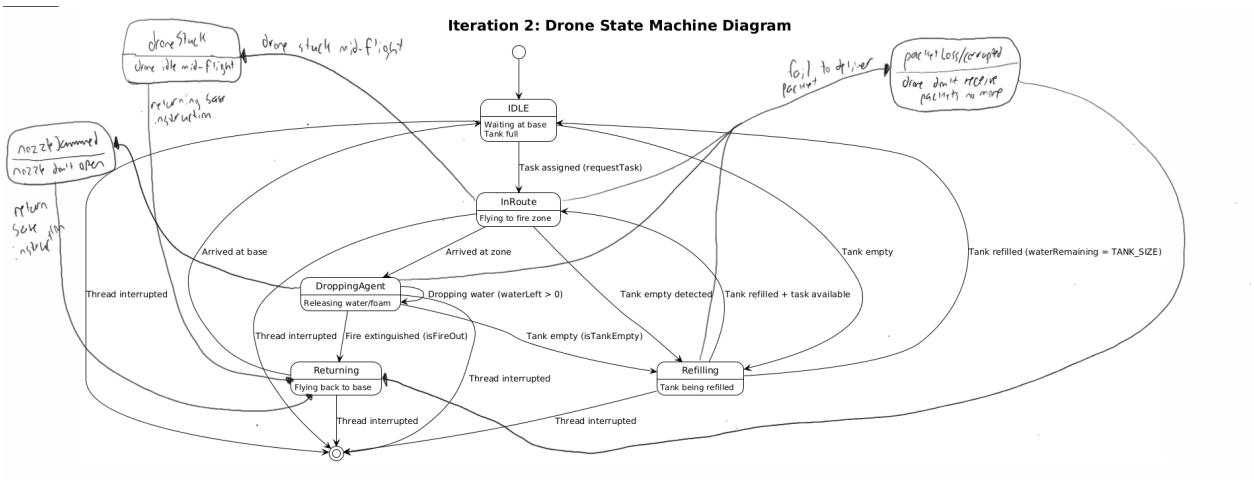
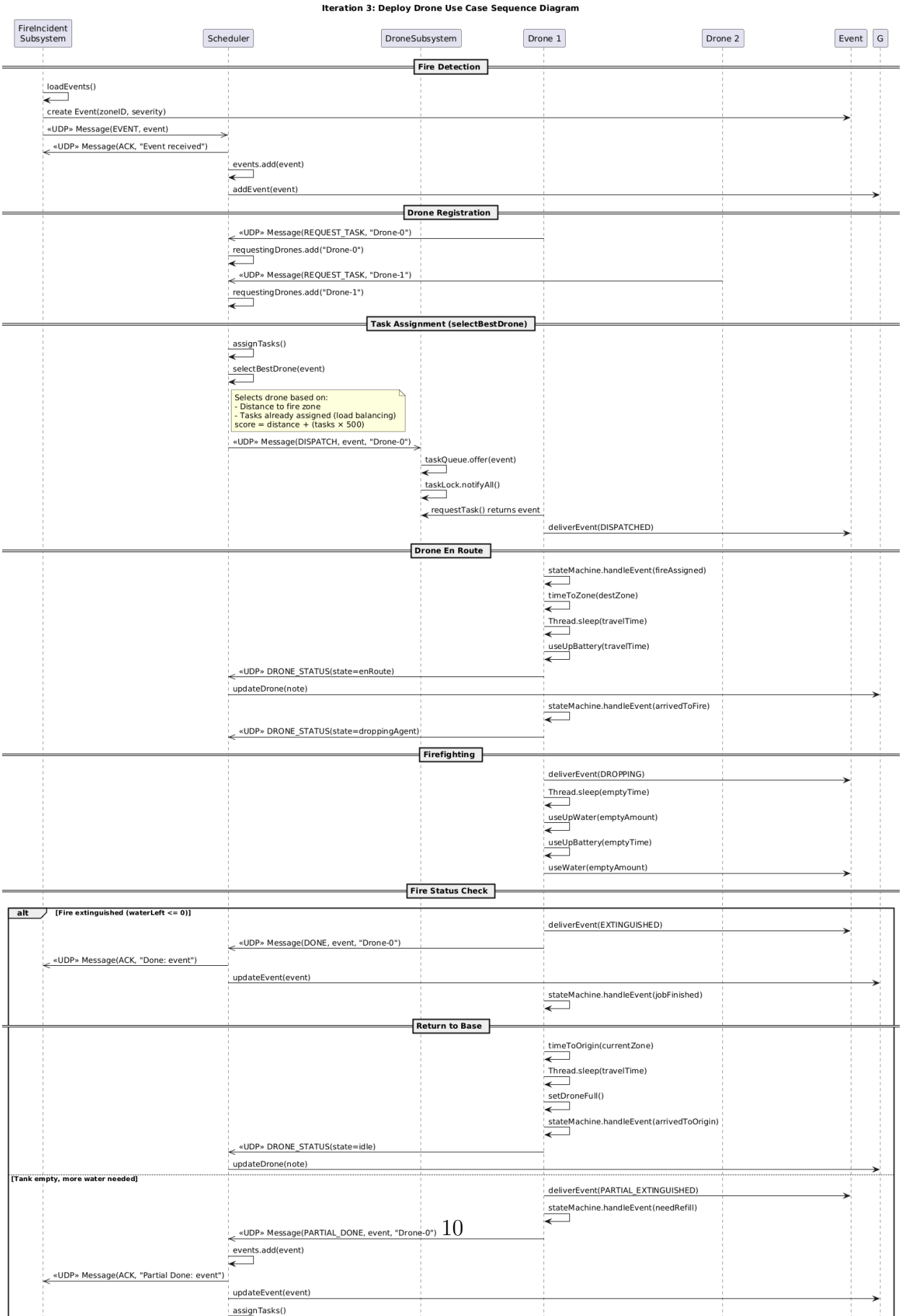


Figure 4: Drone State Machine Diagram updated for Iteration 4 with fault states: drone stuck mid-flight, nozzle jammed, and packet loss/corrupted message.

3.5 Sequence Diagram — Deploy Drone



3.6 Sequence Diagram — Initialization

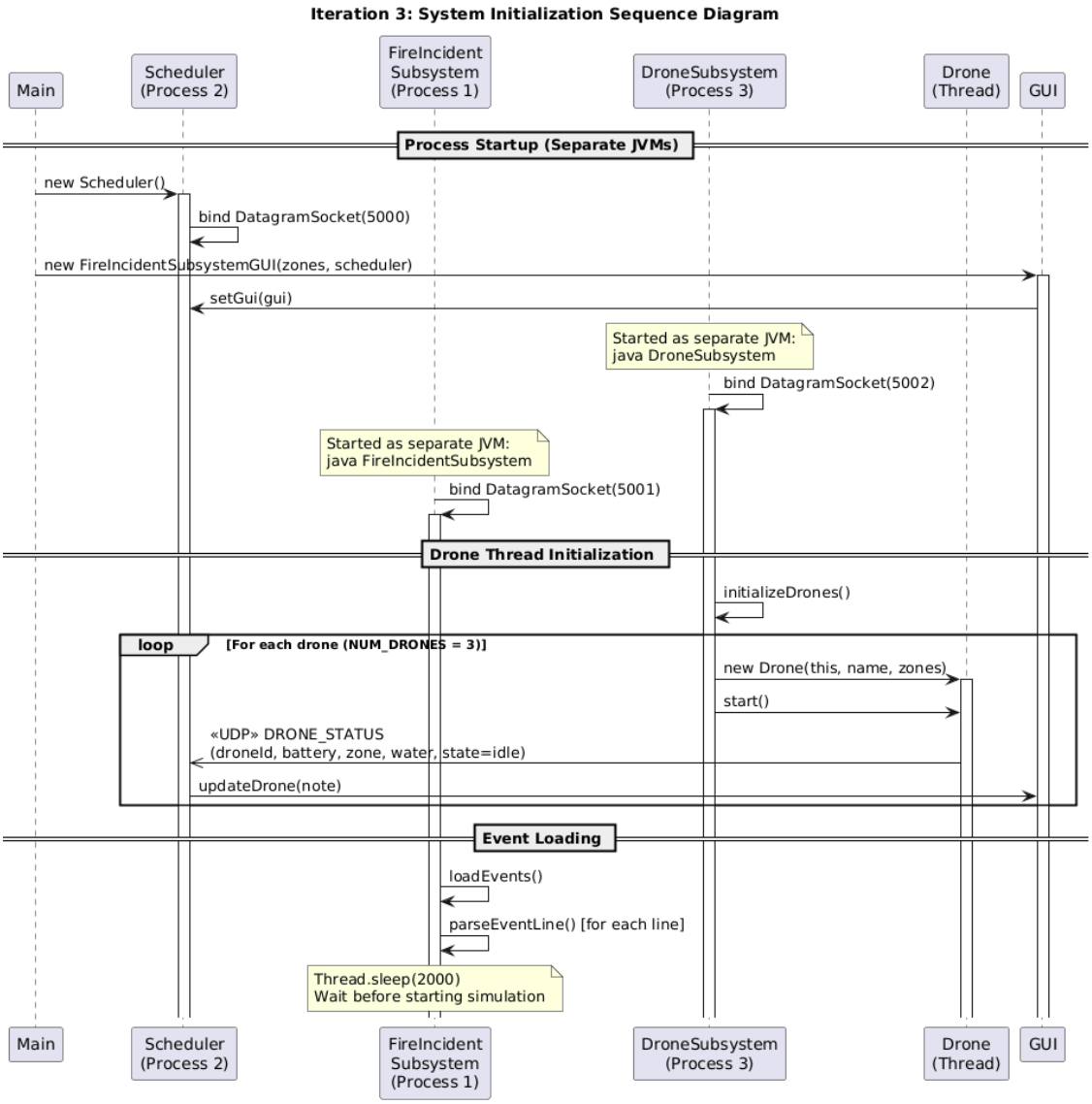


Figure 6: Sequence diagram for system initialization.

3.7 Sequence Diagram — Refill

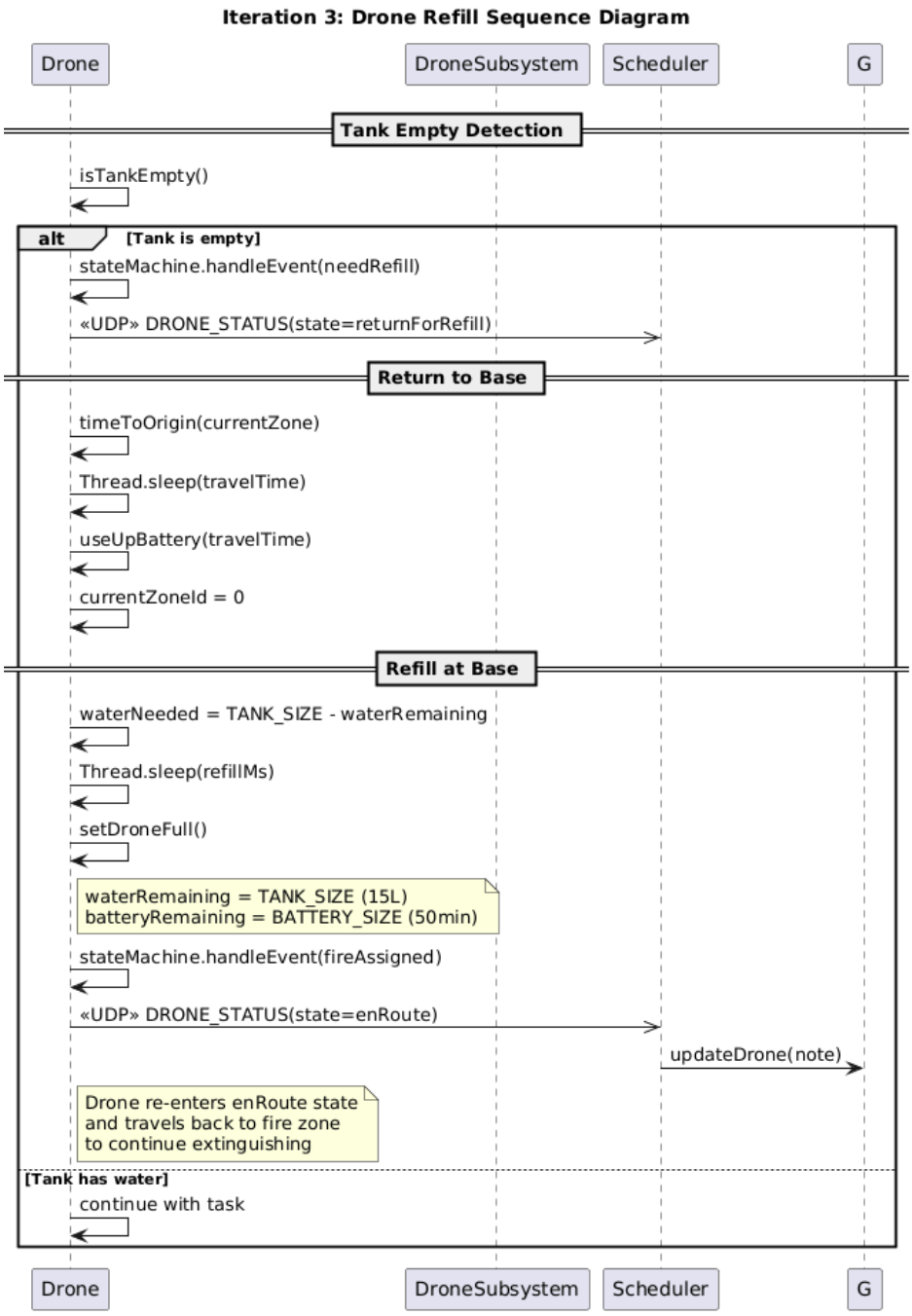


Figure 7: Sequence diagram for the drone refill scenario.

3.8 Sequence Diagrams — Error Scenarios

The error scenario sequence diagrams are included in the timing diagrams (see Section 3.9), which show the full timeline of events across all subsystems for each fault type.

3.9 Timing Diagrams for the Scheduler

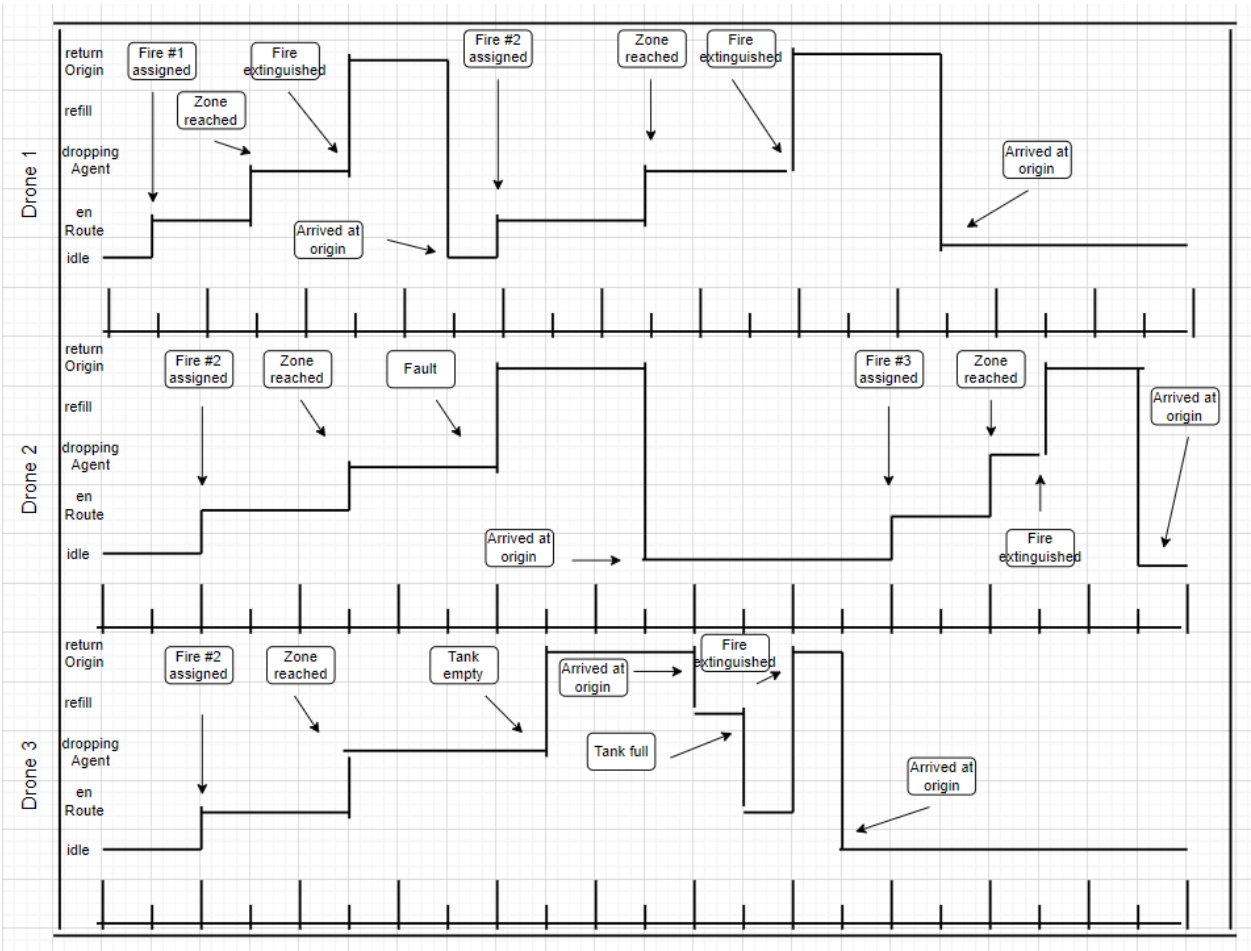


Figure 8: Timing diagram showing three drones operating concurrently. Drone 1 handles two fires. Drone 2 encounters a fault mid-flight and is later reassigned. Drone 3 runs out of agent, refills at base, then returns to finish the fire.

The additional timing diagrams for the three fault scenarios are provided in the accompanying `PI4_TimingDiagrams.pdf`. Each diagram shows the timeline of events across all subsystems with color-coded state bars and message arrows.

- **Diagram 1:** Drone Stuck Mid-Flight — transient fault, timeout detection, Scheduler reroute
- **Diagram 2:** Nozzle Jammed — hard fault, drone offline, Scheduler reroute
- **Diagram 3:** Packet Loss / Corrupted Message — CRC failure, resend, communication failure

4 Fault Handling

4.1 Fault Types

Fault	Type	Recovery	Description
Drone Stuck	Transient	Reset + return to service	Drone fails to arrive within expected timeout. Resets for 3s, returns to base, then re-enters idle pool.
Arrival Sensor	Transient	Reset + return to service	Arrival sensor fails; same recovery as drone stuck.
Nozzle Jammed	Hard	Permanent offline	Nozzle fails during agent drop. Drone returns to base and shuts down permanently for the remainder of the simulation.
Comm Failure	Hard	Permanent offline	CRC validation failure or packet loss exceeding retry limit.

4.2 Fault Injection Methods

4.2.1 CSV Input File

A fifth column in the event CSV specifies the fault type. When a drone picks up that event, the fault is triggered during execution. The drone does not know a fault will occur until it is dispatched.

Listing 1: Sample event file with faults

```
Time,Zone ID,Event type,Severity,Fault
14:03:15,3,FIRE_DETECTED,HIGH,DRONE_STUCK
14:10:00,7,DRONE_REQUEST,MODERATE,NONE
15:20:30,1,FIRE_DETECTED,LOW,NOZZLE_STUCK_OPEN
15:45:00,5,FIRE_DETECTED,HIGH,ARRIVAL_SENSOR_FAILED
16:00:15,2,DRONE_REQUEST,LOW,NONE
```

4.2.2 GUI Injection

The fault injection panel in the GUI allows selecting a specific drone and fault type at runtime. The fault is sent via a `FAULT_INJECT` message from the Scheduler to the Drone-Subsystem, which applies it to the targeted drone.

4.3 Fault Detection Mechanisms

- **Arrival Timeout:** The Scheduler tracks expected arrival times for dispatched drones using the formula: `deadline = animStartTime + (travelMs` $\times$ `2.0) + 1000ms`. If the deadline passes without arrival confirmation, the drone is marked as stuck.

- **CRC32 Validation:** Every `Message` includes a CRC32 checksum computed over the type, ID, sequence number, note, and event payload. If validation fails on receipt, a `RESEND_REQUEST` is sent. After `MAX_RESEND_ATTEMPTS` (1) retries, a communication failure is declared.
- **Sequence Number Gaps:** Messages carry monotonically increasing sequence numbers per sender. Gaps in the sequence on completion-critical messages (DONE, PARTIAL_DONE) indicate dropped packets and trigger a communication failure for the affected drone.
- **Nozzle Fault Detection:** When a drone reports a `PARTIAL_DONE` with a `NOZZLE_STUCK_OPEN` fault type and no change in water level, the Scheduler recognizes the nozzle jam and marks the drone as faulted.

4.4 Scheduler Fault Response

When any fault is detected, the Scheduler performs the following:

1. Marks the drone as failed by adding it to the `failedDrones` set.
2. Removes the drone from the requesting pool.
3. Removes any expected arrival tracking for that drone.
4. Re-queues the assigned fire event (without the fault flag) at the front of the event queue so another drone handles it.
5. For transient faults (stuck, arrival sensor), adds the drone to `transientFaultPendingRepair`. When the drone later reports as idle at base, it is cleared from the failed set and returns to service.
6. For hard faults (nozzle, comm failure), the drone is permanently excluded from task assignment.
7. Updates the GUI to reflect the fault state with appropriate color coding.
8. Immediately calls `assignTasks()` to dispatch a replacement drone if one is available.

4.5 Drone Fault Behavior

4.5.1 Transient Faults (Drone Stuck, Arrival Sensor)

1. The drone detects the fault (via CSV injection or GUI injection).
2. The drone sends its fault state to the Scheduler.
3. The drone freezes for 3 seconds (visible reset period).
4. The drone flies back to base with normal travel animation.

5. Upon arrival at base, the drone resets to idle with full tank and battery.
6. The drone resumes requesting tasks from the Scheduler.

4.5.2 Hard Faults (Nozzle Jammed)

1. The drone detects the nozzle jam when attempting to drop agent.
2. The drone reports a `PARTIAL_DONE` with `NOZZLE_STUCK_OPEN` and unchanged water level.
3. The drone sets its state to `nozzleStuckFault` and flies back to base.
4. Upon arrival at base, `running = false` — the drone thread terminates permanently.
5. The drone is never dispatched again for the remainder of the simulation.

5 Detailed Setup and Test Instructions

5.1 Prerequisites

- Java 17 or higher
- IntelliJ IDEA (recommended)

5.2 Running the Simulation

1. Open the project in IntelliJ IDEA.
2. **Run** `Main.java` — Starts the Scheduler and opens the GUI.
3. **Run** `DroneSubsystem.java` — Starts the Drone Subsystem with all drone threads.
4. **Run** `FireIncidentSubsystem.java` — Reads the CSV and begins sending events.

Important: Start in order: Main → DroneSubsystem → FireIncidentSubsystem.

5.3 Testing Fault Scenarios

5.3.1 CSV-Based Fault Injection

Edit `Sample_event_file.csv` to include faults in the 5th column. Valid values: `NONE`, `DRONE_STUCK`, `ARRIVAL_SENSOR_FAILED`, `NOZZLE_STUCK_OPEN`.

Expected behavior:

- **DRONE_STUCK**: Drone gets stuck mid-flight, resets after 3 seconds, returns to base, re-enters service. Scheduler dispatches replacement.
- **NOZZLE_STUCK_OPEN**: Drone's nozzle jams during drop. Returns to base and goes permanently offline. Scheduler dispatches replacement.
- **ARRIVAL_SENSOR_FAILED**: Same transient recovery as DRONE_STUCK.

5.3.2 GUI-Based Fault Injection

1. In the **Fault Injection** panel (bottom-right of GUI), select a drone and fault type.
2. Click **Inject Fault**.
3. Observe the drone's state change in the table, map, and event log.

5.4 Unit Tests

- `DroneStateMachineJUnitTest.java` — State transition tests
- `FireIncidentSubsystemJUnitTest.java` — CSV parsing tests
- `UdpCommunicationJUnitTest.java` — Message serialization and UDP tests
- `PacketLossDetectionJUnitTest.java` — CRC32, sequence gap, resend tests
- `FaultDetectionJUnitTest.java` — Fault injection and recovery tests

5.5 Test Input Files

Event file (`Sample_event_file.csv`):

```
Time,Zone ID,Event type,Severity,Fault
14:03:15,3,FIRE_DETECTED,HIGH,DRONE_STUCK
14:10:00,7,DRONE_REQUEST,MODERATE,NONE
15:20:30,1,FIRE_DETECTED,LOW,NOZZLE_STUCK_OPEN
15:45:00,5,FIRE_DETECTED,HIGH,ARRIVAL_SENSOR_FAILED
16:00:15,2,DRONE_REQUEST,LOW,NONE
```

Zone file (`sample_zone_file.csv`):

```
Zone ID,Zone Start,Zone End
1,(0,0),(600,500)
2,(700,0),(1300,500)
3,(0,600),(600,1100)
4,(700,600),(1300,1100)
5,(0,1200),(600,1700)
6,(700,1200),(1300,1700)
7,(300,1800),(900,2300)
```

6 Results from Measurements

The Scheduler generates a metrics report at the end of each simulation, saved to `simulation_metrics_*.txt`

Table 1: Simulation overview (3 drones, 5 events, simulationSpeed=10)

Metric	Value
Incidents detected	5
Incidents extinguished	5
First fire simulated timestamp	14:03:15
Last fire extinguished simulated timestamp	15:45:00
Total simulation time	1m 51s 305ms
First incident to last extinguished	1m 41s 884ms
Total dispatch distance	5939.39

Table 2: Per-incident metrics

Time	Zone	First Response	Detect → Extinguish	Dispatch Dist.
14:03:15	3	0m 00s 136ms	0m 32s 077ms	901.39
14:10:00	7	0m 00s 167ms	0m 46s 032ms	2136.00
15:20:30	1	0m 00s 135ms	0m 07s 182ms	390.51
15:45:00	5	0m 00s 033ms	0m 40s 801ms	1480.71
16:00:15	2	0m 00s 095ms	0m 10s 464ms	1030.78

Table 3: Per-drone metrics

Drone	Idle Time	Flight Time	Dispatch Dist.	Fires	Water (L)
Drone-0	0m 26s 581ms	0m 54s 578ms	2382.10	2	60.00
Drone-1	0m 54s 641ms	0m 46s 536ms	2136.00	1	20.00
Drone-2	1m 26s 661ms	0m 14s 520ms	1421.29	2	20.00

7 Design Reflection

7.1 What We Like

- **Clean separation of concerns:** Each subsystem runs as an independent process with well-defined UDP message interfaces, allowing team members to work on different subsystems concurrently.
- **Robust fault handling:** CRC32 validation, sequence number tracking, resend protocols, and arrival timeouts provide multiple layers of fault detection. Transient faults recover automatically, while hard faults permanently remove drones — matching the professor's specifications.

- **GUI:** Animated drone positions, flight paths, pulsing fault indicators, color-coded fault states, a fault injection panel, and a timestamped event log provide comprehensive situational awareness.
- **Configurable simulation:** Drone count, simulation speed, zone layout, and agent capacity are easily adjustable for different test scenarios.
- **Automatic metrics:** The Scheduler collects and reports detailed performance metrics including per-drone and per-incident statistics.

7.2 What Should Be Redone

- **Scheduling algorithm:** The current distance + task-count heuristic could be improved with battery-aware scheduling, severity-based priority queues, and multi-drone dispatch for high-severity fires.
- **Collision avoidance:** The system does not account for potential drone collisions. Path planning or time-offset dispatching would be a valuable addition.
- **Dynamic map scaling:** The zone map uses fixed scaling. Supporting zoom/pan and dynamically adapting to different zone configurations would improve usability.